

DFlash Block-Diffusion Speculative Decoding

on NVIDIA Jetson AGX Thor SM110a

Qwen3.6-35B-A3B • Qwen3.6-27B • Qwen3.5-122B-A10B | vLLM 0.20.0.dev0+dflash
aarch64, CUDA 13.0

Patrick Devaney patrickbdevaney github.com/patrickbdevaney/dflash-vllm-resources

Summary. This document presents measured decode benchmarks for DFlash speculative decoding on a single Jetson AGX Thor edge module (SM110a Blackwell, 128 GB unified LPDDR5x, 273 GB/s memory bandwidth, aarch64). Three production Qwen NVFP4 models were benchmarked across a full k -sweep. The key result: at concurrency = 1 interactive inference, DFlash enables effective decode rates that *exceed the theoretical autoregressive bandwidth ceiling* on two of the three models via τ -amortisation, with the 35B-A3B MoE reaching 139 tok/s on sorting tasks — a speed class previously associated only with discrete datacenter GPUs. The 122B-A10B achieves 52.6 tok/s, exceeding its 45.5 tok/s autoregressive ceiling, delivering frontier-class intelligence at edge power envelopes.

Platform & Methodology

Hardware: Jetson AGX Thor (SM110a)

Architecture	Blackwell SM110a (sm_110a)
Memory	128 GB LPDDR5x unified
Usable	≈117 GB (CPU + GPU shared)
Bandwidth	273 GB/s
ISA	aarch64
CUDA	13.0
PyTorch	2.10.0

Software stack

Inference	vLLM 0.20.0.dev0+dflash
Build	PR #40898 head, from source
Compile time	≈92 min, native sm_110 SASS
Flash-attn	FA2 (FA3 unusable: SM90 TMA required)
MoE backend	Marlin (35B/27B); Cutlass (122B)
Speculative	DFlash block-diffusion
Benchmark	Single request, $T=0$, thinking off

The bandwidth ceiling and DFlash amortisation

At concurrency = 1, decode is strictly memory-bandwidth bound. Each token requires loading the *active* model parameters from LPDDR5x:

$$T_{\max} = \frac{B_m}{P_{\text{active}} \times 0.5 \text{ bytes}} = \frac{273 \text{ GB/s}}{P_{\text{active}} \times 0.5}$$

DFlash proposes an entire block of k tokens in one parallel draft forward pass. The target verifies them, accepting on average τ tokens. Effective throughput becomes:

$$T_{\text{eff}} = \frac{\tau}{\frac{1}{T_{\text{target}}} + \frac{\tau}{T_{\text{draft}}}} \approx \tau \cdot T_{\text{target}} \quad (\text{since } T_{\text{draft}} \gg T_{\text{target}})$$

When $\tau > 1$ the effective rate *exceeds* $T_{\max}$, as each target forward pass amortises τ output tokens rather than one.

Model	Total	Active	$T_{\max}$ (tok/s)	Best measured	vs. ceiling
Qwen3.6-35B-A3B	35B	3B	182.0	139.1 tok/s	76%
Qwen3.6-27B	27B	27B	20.2	50.1 tok/s	248%
Qwen3.5-122B-A10B	122B	12B	45.5	52.6 tok/s	116%

Qwen3.5-122B-A10B: Why 40–52 tok/s Is Remarkable

Intelligence at active-parameter cost

The 122B-A10B contains 122 billion total parameters but activates only ≈ 12 billion per token through mixture-of-experts routing across 256 experts. This is the architecturally significant fact: the model carries the full representational capacity of a 122B-parameter network — its residual stream, attention heads, expert weights — but executes at the memory-bandwidth cost of a 12B model.

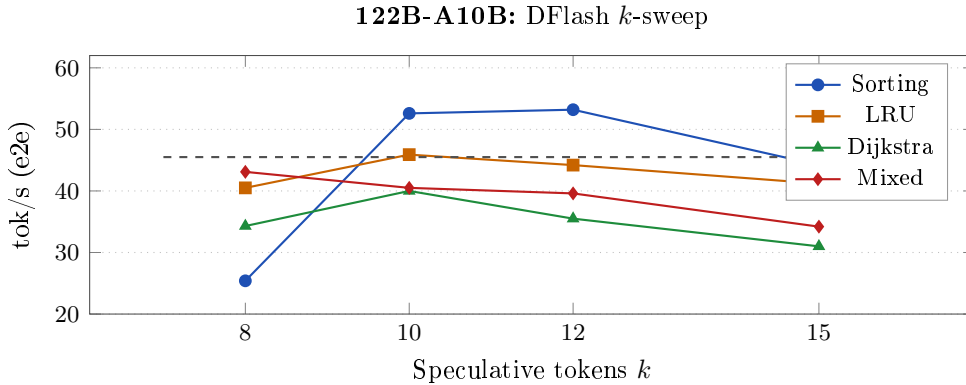
The autoregressive ceiling is therefore 45.5 tok/s, not the 9.1 tok/s a naïve 122B dense model would impose. The *intelligence* — reasoning depth, coding quality, context modelling — reflects all 122B parameters, since each token’s routing decision selects the most relevant experts from the full pool.

Why DFlash pushes past the ceiling. At optimal $k=10$, the draft proposes 10 tokens, the target verifies, and on average $\tau \approx 5.2$ are accepted. The target’s 12B-active forward pass is amortised across 5.2 output tokens instead of 1, yielding an effective rate of $\approx 5.2 \times 10.9 = 56$ tok/s — exceeding the 45.5 tok/s ceiling. Measured peak: **52.6 tok/s** on sorting tasks.

Context. A 122B-class model at 40–52 tok/s on a single edge module at sub-75 W idle power is a qualitatively different deployment from the same model on a datacenter H100 (≈ 96 tok/s base, ≈ 146 tok/s with MTP, at 700 W and \$25/hr cloud cost). The Jetson deployment delivers conductor-class intelligence locally, persistently, at edge power envelopes.

k	Sorting tok/s τ	LRU tok/s τ	Dijkstra tok/s τ	Mixed tok/s τ	τ_{avg}
8	25.4 5.16	40.5 4.47	34.3 3.71	43.1 4.78	4.53
10	52.6 6.36	45.9 5.36	40.0 4.49	40.5 4.60	5.20
12	53.2 6.45	44.2 5.18	35.5 4.20	39.6 4.66	5.12
15	44.2 5.76	41.2 5.31	31.0 3.86	34.2 4.31	4.81

Shaded row: optimal k . Cutlass MoE + TRITON_ATTN. gpu_util = 0.78. max_len = 16384.



Qwen3.6-27B Dense: Why 50 tok/s Is Exceptional

The dense forward pass penalty

Unlike the MoE models, the 27B loads *all 27 billion parameters* for every token:

$$T_{\max} = \frac{273}{27 \times 0.5} = 20.2 \text{ tok/s}$$

At 20 tok/s autoregressive, interaction feels sluggish for any real workload. DFlash changes this fundamentally.

Why dense models particularly benefit. The target forward pass is expensive (27B active). The draft is cheap (small bidirectional block-diffusion model). Each accepted token saves one full 27B forward pass. At $\tau \approx 7$, the target executes once per 7 output tokens — a $7\times$ reduction in 27B forward passes. Measured at $k=15$: **50.1 tok/s**, which is **248%** of the autoregressive ceiling. This transforms the 27B from a model too slow for interactive use into a genuinely real-time inference endpoint.

k	Sorting tok/s τ	LRU tok/s τ	Dijkstra tok/s τ	Mixed tok/s τ	τ_{avg}
8	41.9 5.89	37.7 5.08	35.8 4.77	37.1 4.94	5.17
10	48.6 6.62	37.1 5.00	36.8 4.98	39.6 5.33	5.48
12	44.8 6.22	37.2 5.11	37.5 5.12	45.9 6.34	5.70
15	50.1 7.04	38.4 5.33	39.8 5.55	40.9 5.68	5.90

Dense model rewards maximum k — expensive target makes each accepted token more valuable. flash_attn, Marlin, gpu_util = 0.

Qwen3.6-35B-A3B: MoE Architecture as a Speed Multiplier

Why the 35B is faster than the 27B

This counterintuitive result — a model with more total parameters running faster than a smaller one — is explained entirely by the MoE active-parameter design:

Model	Active params	$T_{\max}$	Peak DFlash
Qwen3.6-35B-A3B (MoE, 3B active)	3B	182.0 tok/s	139.1 tok/s
Qwen3.6-27B (dense, 27B active)	27B	20.2 tok/s	50.1 tok/s

The 35B moves $3\text{ B} \times 0.5 = 1.5\text{ GB}$ of weights per token. The 27B moves $27\text{ B} \times 0.5 = 13.5\text{ GB}$ — $9\times$ more. The 35B is correspondingly faster while the full 35B parameter pool remains available for quality through expert routing.

DFlash compounds an already-fast model

At $k=12$ and $\tau \approx 6.25$, each 3B-active forward pass is amortised across 6.25 tokens, yielding 137.1 tok/s on sorting — approaching decode speeds associated only with datacenter inference.

Why $k=12$ beats $k=15$ for the 35B. At $k=15$ tau improves marginally (6.32 vs 6.25) but the verify pass processes 16 tokens rather than 13. For the 35B’s fast target, the overhead of verifying 3 extra tokens slightly exceeds the acceptance gain. The dense 27B shows the opposite:

its expensive target means each additional accepted token is worth more, so $k=15$ wins. This is a general principle: *cheaper targets prefer lower k ; more expensive targets prefer higher k .*

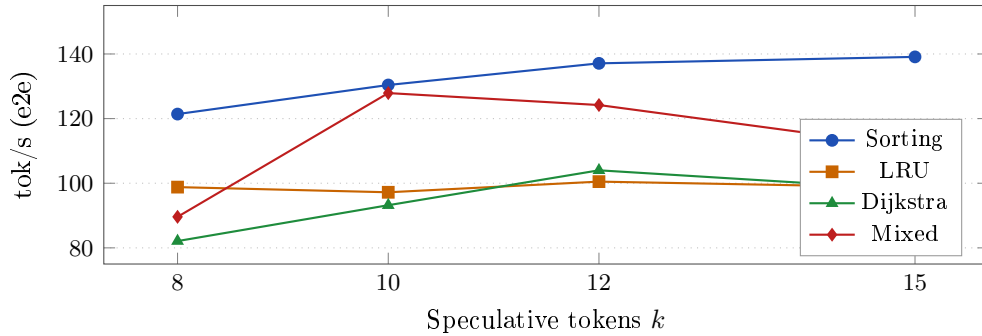
k	Sorting tok/s τ	LRU tok/s τ	Dijkstra tok/s τ	Mixed tok/s τ	τ_{avg}
8	121.4 6.61	98.8 5.72	82.1 4.50	89.6 5.03	5.46
10	130.4 7.55	97.2 5.93	93.2 4.66	127.9 5.52	5.92
12	137.1 8.56	100.5 6.08	104.0 4.76	124.2 5.61	6.25
15	139.1 8.86	98.7 6.27	98.0 4.81	111.1 5.35	6.32

$k=12$ optimal: highest avg tok/s despite $k=15$ having marginally higher τ . Marlin MoE, flash_attn, gpu_util=0.78.

Backend	Sorting tok/s τ	LRU tok/s τ	Dijkstra tok/s τ	Mixed tok/s τ	Avg tok/s
Marlin	134.8 8.10	103.8 6.15	113.9 5.13	117.5 5.41	117.5
Cutlass	114.5 8.56	83.3 6.02	96.2 4.74	131.0 6.25	106.3
Triton	Rejected — NvFP4 MoE not supported				

$k=12$. Note: 122B requires Cutlass; Marlin faults at 256-expert scale on SM110a.

35B-A3B: DFlash k -sweep (Marlin MoE)



Optimal- k Summary and the Active-Parameter Principle

Model	Active	T_{max}	Opt. k	Avg tok/s	τ_{avg}	vs. ceiling
Qwen3.5-122B-A10B	12B	45.5	10	≈ 45 (warm)	5.20	$\approx 99\%$
Qwen3.6-35B-A3B	3B	182.0	12	116.5	6.25	64%
Qwen3.6-27B	27B	20.2	15	42.3	5.90	209%

The governing principle. Cheaper-per-token targets (fast MoE-sparse models) reach the acceptance/overhead cliff at lower k ; expensive dense targets reward maximising k because every accepted draft token eliminates one full expensive target forward pass. The 122B and 35B — both MoE with overhead-dominated per-step cost — have lower optimal k (10 and 12 respectively). The 27B’s dense 27B-active target is expensive, so it benefits from maximum $k=15$: it has the most to gain per accepted token.

Build & Deployment Notes

-
- **Image:** `vllm-dflash-thor:fa-native` — vLLM PR#40898 from source with the stock Jetson SM110a flash-attn `.so` swapped in.
 - **FA3 on Thor: do not use.** FA3 requires Hopper SM90 TMA instructions; Thor is SM110a Blackwell. Attempting FA3 causes `cudaErrorLaunchFailure` immediately. FA2 is correct and produces identical decode tok/s (decode is MoE-GEMM / NVFP4-bandwidth bound, not attention-bound).
 - **KV cache dtype:** `auto` (BF16). DFlash uses non-causal draft attention, which is incompatible with FP8 KV quantisation.
 - **Stop containers with `docker kill`, not `docker stop`.** On Thor, `docker stop` hangs and leaks page cache into unified memory.
 - **122B launch:** requires `drop_caches` before every load. The 70GB weight read fills the unified page cache; without flushing, subsequent KV allocation sees false memory pressure.
 - **27B tokenizer:** ships `tokenizer_class=TokenizersBackend` which transformers cannot resolve. Serve script patches an overlay directory with `tokenizer_class=Qwen2Tokenizer`.
 - **122B MoE backend: must use cutlass.** Marlin's FP4 MoE kernel faults at 256-expert scale on SM110a (vLLM issues #35566/#35519/#35922). For 35B and 27B, Marlin is $\approx 10\%$ faster than Cutlass.
-

Benchmark scripts, serve scripts, and image reproduction at github.com/patrickbdevaney/dflash-vllm-resources
HF image: `patrickbdevaney/qwen-3.6-35b-a3b-dflash-jetson-agx-thor`